

# 线性关系与存储表示

同一逻辑序列怎样换表示，操作为什么随之改变

408 · 数据结构 Topic DS-01

母问题：逻辑线性关系怎样被有限存储兑现？

线性表不是“数组或链表的名字”，而是一组对元素顺序和操作结果的契约。手册的生成链是：

Linear Relation → Operation Contract → Representation  
→ Local Update → Invariant → Cost Vector

箭头先表示建模顺序，再表示一次操作中信息和状态如何流动：先固定逻辑对象，才知道某种物理布局是否值得维护。不同表示必须给出相同的抽象结果，但不必承担相同的移动、寻址和额外空间成本。

本册定位与 Owner 边界

- **Owns**：线性表 ADT、顺序表、单链表、双链表、静态链式表示；数组的多维地址映射与特殊矩阵压缩；插入/删除/按位访问/按值查找的状态变化与边界。
- **Uses**：Atlas Foundation 的问题规模、基本操作、最坏/平均/摊还成本和成本向量语言。
- **Outputs**：向 DS02 提供“底层顺序/链接表示”，向 DS09 提供“有序序列上的访问与插入代价”；栈、队列、字符串和索引只调用本册表示，不复制本册机制。
- **Stops**：受限端点语义归 DS02；KMP 归 DS03；树/图节点结构归 DS04/DS07；BST、AVL、B/B+ 的有序索引机制归 DS09。

## 目录

|     |                       |   |
|-----|-----------------------|---|
| 1   | 先把“表”与“存储”分开          | 3 |
| 2   | 顺序表：地址映射换取按位访问        | 3 |
| 2.1 | 长度、容量与合法状态            | 3 |
| 2.2 | 插入与删除是区域移动            | 3 |
| 3   | 多维数组与特殊矩阵：坐标如何映射为线性偏移 | 4 |
| 3.1 | 压缩存储：先找独立元素，再建立可逆编号   | 4 |
| 4   | 链式表示：指针关系换取局部修改       | 4 |
| 4.1 | 链表的不变量                | 4 |
| 4.2 | 局部插入的完整生命周期           | 5 |
| 5   | 同一操作的两套成本向量           | 5 |
| 6   | 生成性例子：把“按值删除”拆成两个阶段   | 5 |
| 7   | 代码把模型变成可执行状态转移        | 6 |

|                                              |    |
|----------------------------------------------|----|
| 7.1 顺序表：先保证容量，再从后向前移动 . . . . .              | 6  |
| 7.2 单链表：局部 $O(1)$ 从“已知前驱”开始 . . . . .        | 7  |
| 7.3 按值删除：Locate 与 Update 在代码中也必须分层 . . . . . | 8  |
| 7.4 不变量检查与测试不是生产操作的成本 . . . . .              | 8  |
| 8 空、首、尾与重复值不是补丁                              | 9  |
| 9 概念边界：把相似说法还原成判据                            | 10 |
| 10 做题控制协议与压缩复原                               | 10 |
| 11 全科坐标与跨册交接                                 | 10 |
| 12 来源处理                                      | 10 |

# 1 先把“表”与“存储”分开

抽象线性表  $L = (a_1, a_2, \dots, a_n)$  只承诺元素的线性次序、长度和允许操作。它不承诺元素在内存中连续，也不承诺一次操作一定只访问一个地址。若把逻辑位置记为  $i$ ，必须先区分：

| 层次   | 要回答的问题                 | 不能直接推出的结论            |
|------|------------------------|----------------------|
| 逻辑关系 | 谁是第 $i$ 个元素？插入后次序怎样？   | 元素一定相邻               |
| 操作契约 | 按位访问、按值查找、插入、删除的输入是什么？ | 每种操作都同样便宜            |
| 物理表示 | 数据、长度、容量、指针和空闲单元怎样保存？  | 只看元素字段就能判断状态         |
| 成本模型 | 计数对象是比较、移动、指针跳转还是 I/O？ | 一个 $O(\cdot)$ 覆盖所有场景 |

最小反例：同一个“插入第  $k$  个”

对逻辑表  $[a, b, c, d]$  在位置 3 插入  $x$ ，抽象结果只有一个： $[a, b, x, c, d]$ 。顺序表需要把  $c, d$  向后移动；链表只需让  $x$  接入正确的指针链，但前提是已经拿到插入点的前驱。若题目只给关键字而不给位置，寻找前驱的成本又回来了。

## 2 顺序表：地址映射换取按位访问

顺序表把元素放入一段可按下标解释的连续区域。设首地址为  $B$ ，每个元素占  $w$  个存储单元，则零基础下标  $i$  的地址为

$$\text{addr}(A[i]) = B + i w.$$

这条式子不是“数组 API 口诀”，而是顺序表能在  $\Theta(1)$  时间定位第  $i$  个元素的原因：位置到地址是一次算术映射，不需要沿中间元素搜索。

### 2.1 长度、容量与合法状态

动态顺序表至少维护  $(data, length, capacity)$ 。合法状态包括

$$0 \leq length \leq capacity, \quad 0 \leq i < length \Rightarrow A[i] \text{ 是有效元素.}$$

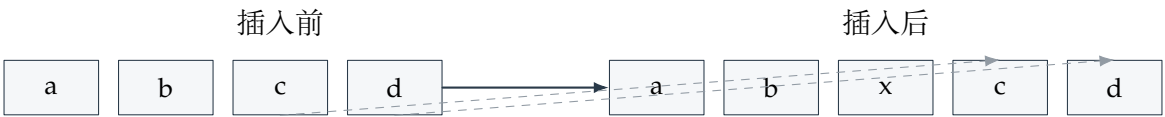
插入前先检查  $length < capacity$ ；若需要扩容，旧元素必须按原次序复制到新区域，再改变容量和写入位置。扩容后的单次复制可能是  $\Theta(n)$ ，但若容量按固定倍数增长，连续追加的总复制量可摊还到每次  $O(1)$ ；这里的“摊还”不是把一次最坏成本改写成  $O(1)$ 。

### 2.2 插入与删除是区域移动

在长度为  $n$  的表中，在逻辑位置  $k$  ( $0 \leq k \leq n$ ) 插入：

$$\text{检查容量} \rightarrow A[n-1] \rightsquigarrow A[k] \text{ 右移} \rightarrow A[k] \leftarrow x \rightarrow length++.$$

需要移动  $n - k$  个元素；在头部插入为  $\Theta(n)$ ，尾部追加在有剩余容量时为  $\Theta(1)$ 。删除位置  $k$  则把区间  $A[k+1 \dots n-1]$  左移，移动  $n - k - 1$  个元素，再递减长度。覆盖或清理被删除槽位是实现选择，但不能让无效槽位继续被逻辑遍历访问。



### 3 多维数组与特殊矩阵：坐标如何映射为线性偏移

数组的维数是逻辑坐标的个数，内存地址仍是一维序列。行优先、列优先不是两种数组对象，而是两种线性化函数。设二维数组  $A[L_1..U_1][L_2..U_2]$ ，每个元素占  $w$  个存储单元， $N_1 = U_1 - L_1 + 1$ ， $N_2 = U_2 - L_2 + 1$ ，则

$$\text{addr}_{\text{row}}(A[i][j]) = B + ((i - L_1)N_2 + (j - L_2))w,$$

$$\text{addr}_{\text{col}}(A[i][j]) = B + ((j - L_2)N_1 + (i - L_1))w.$$

两式都从同一问题生成：目标坐标之前，线性序列中已经排了多少个元素？下界不从 0 或 1 开始时，必须先减去逻辑下界；只背  $in + j$  会在非零下界题中错位。

对  $d$  维数组，行优先偏移可递推为

$$\text{offset} = (((i_1 - L_1)N_2 + (i_2 - L_2))N_3 + \cdots)N_d + (i_d - L_d),$$

本质是混合进位制，不需要另背展开式。

#### 3.1 压缩存储：先找独立元素，再建立可逆编号

特殊矩阵只有在未存部分可由结构规则恢复时才能压缩。先确定独立元素集合，再为它建立连续编号  $k = f(i, j)$ ：

- 对称矩阵只存一个三角区，因为  $a_{ij} = a_{ji}$ ；访问另一半时先交换坐标再编号。
- 上/下三角矩阵存一个三角区和一个公共常数；另一三角区的元素不占独立槽位。
- 对角、带状矩阵只存满足带宽约束的坐标；带外元素由题设常数恢复。
- 稀疏矩阵通常保存非零三元组  $(\text{row}, \text{column}, \text{value})$ ；它换掉了连续随机定位能力，查找需额外搜索或索引。

例如按行存储下三角矩阵（含主对角线），零基坐标满足  $0 \leq j \leq i < n$ 。第  $i$  行之前已有  $i(i+1)/2$  个元素，所以

$$k = \frac{i(i+1)}{2} + j.$$

公式证据是“前面完整行的长度和 + 当前行内偏移”。若改用一基下标或上三角按列存储，应重新数前置元素，不能机械平移。

#### 数组映射题的停止条件

先写合法坐标域、存储顺序和独立元素集合；再数目标元素之前的槽位，最后乘元素宽度并加首地址。编号必须落在  $[0, m-1]$ ，其中  $m$  是实际槽数；若两个本应独立的坐标映到同一编号，映射就不合法。

### 4 链式表示：指针关系换取局部修改

单链表节点保存  $(\text{value}, \text{next})$ ，表对象通常保存头指针和可选的尾指针、长度。逻辑相邻不等于物理相邻： $a_i$  的后继由指针给出，而不是由地址加  $w$  得到。双链表再保存  $\text{prev}$ ，换取从后继回到前驱的能力。

#### 4.1 链表的不变量

对无环单链表，必须保持：

1. 头指针要么为空，要么指向第一个有效节点；

2. 从头沿 *next* 访问，每个有效节点至多出现一次，最终到达空指针；

3. 若维护 *tail*，则 *tail* 是最后节点且 *tail.next* = *null*；

4. 若维护 *length*，它等于从头可达节点数。
- 双链表还要保持相邻互逆： $u.next = v \Rightarrow v.prev = u$ 。任一方向只改一条边都会暂时破坏结构，后续操作可能沿错误方向丢失节点。

4.2 局部插入的完整生命周期

已知前驱 *p*，在其后插入新节点 *x* 的最小动作是：

$$x.next \leftarrow p.next, \quad p.next \leftarrow x.$$

若 *p* 原来是尾节点，还要更新 *tail*；若向空表插入，还要同时更新 *head*。删除节点 *x* 时，必须先拿到前驱 *p*，令  $p.next \leftarrow x.next$ ，再处理尾指针、长度和资源回收。“指针修改  $O(1)$ ”只描述已定位节点后的局部动作，不包括按关键字寻找节点。

5 同一操作的两套成本向量

不要用一个复杂度数字抹平表示差异。对典型操作，至少记录“是否已知位置/前驱、比较数、移动或跳转数、额外空间和最坏边界”：

| 操作            | 顺序表                       | 单链表                       | 成本解释            | 关键前提         |
|---------------|---------------------------|---------------------------|-----------------|--------------|
| 按位访问 <i>i</i> | $\Theta(1)$               | $\Theta(n)$               | 地址映射 vs 指针跳转    | <i>i</i> 合法  |
| 按值查找          | $\Theta(n)$               | $\Theta(n)$               | 逐项比较            | 无额外索引        |
| 已知位置插入        | $\Theta(n)$               | $\Theta(1)$               | 区间移动 vs 改两条边    | 链表已知前驱       |
| 头部插入/删除       | $\Theta(n)$ / $\Theta(n)$ | $\Theta(1)$ / $\Theta(1)$ | 顺序表整体移动；链表改头指针  | 正确维护 head    |
| 尾部追加          | $\Theta(1)$ 摊还            | $\Theta(1)$               | 容量复制 vs tail 指针 | 无扩容/已维护 tail |

空间也要分开：顺序表可能有未使用容量，链表每个节点额外保存指针；两者的逻辑元素空间都是  $\Theta(n)$ ，但常数、局部性和内存管理成本不同。

6 生成性例子：把“按值删除”拆成两个阶段

给定表  $L = [7, 2, 9, 2, 5]$ ，删除第一次出现的 2。这不是一个动作，而是：

1. **Locate**：从头按逻辑次序比较，找到位置  $k = 1$ ；

2. **Update**：顺序表把  $A[2 \dots 4]$  左移，链表把前驱的后继越过目标节点；

3. **Repair**：更新长度、尾指针、空槽或回收节点；

4. **Verify**：再次遍历，确认次序为  $[7, 9, 2, 5]$  且所有结构不变量成立。

两种表示的 **Locate** 都可能是  $\Theta(n)$ ；因此不能看到链表的局部删除就断言“链表删除总是  $O(1)$ ”。真正改变的是 **Update** 的成本和维护方式。

## 7 代码把模型变成可执行状态转移

代码不是本册结论之后的语法附录。它必须逐项兑现操作契约、状态字段、不变量修复和边界分支。完整 C++17 实现位于 `code/ds01_linear_list.hpp`；下面的代码块直接从该文件抽取，因此正文与可运行实现共享同一来源。

### 7.1 顺序表：先保证容量，再从后向前移动

Listing 1: 顺序表插入与删除的核心转移

```

29 void insert(std::size_t index, int value) {
30     if (index > length_) {
31         throw std::out_of_range("insert position is outside [0, length]");
32     }
33
34     ensure_capacity(length_ + 1);
35     for (std::size_t i = length_; i > index; --i) {
36         data_[i] = data_[i - 1];
37     }
38     data_[index] = value;
39     ++length_;
40 }
41
42 int erase(std::size_t index) {
43     require_index(index);
44     const int removed = data_[index];
45     for (std::size_t i = index; i + 1 < length_; ++i) {
46         data_[i] = data_[i + 1];
47     }
48     --length_;
49     return removed;
50 }

```

插入循环必须从 *length* 向 *index* 逆向移动；若正向覆盖，尚未复制的旧元素会被破坏。*ensure\_capacity* 在移动前执行，保证  $length + 1 \leq capacity$ 。删除则正向覆盖被删位置，并在移动完成后递减长度。两段代码分别把“不覆盖未迁移数据”和“有效区恰为  $[0, length)$ ”落成了执行顺序。

扩容本身是一次显式状态迁移：

Listing 2: 容量倍增、复制和所有权切换

```

76 void ensure_capacity(std::size_t required) {
77     if (required <= capacity_) {
78         return;
79     }
80
81     std::size_t new_capacity = capacity_;
82     while (new_capacity < required) {
83         new_capacity *= 2;
84     }

```

```

85
86     auto replacement = std::make_unique<int[]>(new_capacity);
87     std::copy_n(data_.get(), length_, replacement.get());
88     data_ = std::move(replacement);
89     capacity_ = new_capacity;
90 }

```

旧数据完全复制后才切换 `data_` 与 `capacity_`，因此异常或中途失败不会留下“容量已经变大但数据还没搬完”的半合法状态。一次扩容仍为  $\Theta(n)$ ；只有分析连续追加序列时，倍增策略才给出摊还  $O(1)$ 。

## 7.2 单链表：局部 $O(1)$ 从“已知前驱”开始

Listing 3: 已知前驱后的插入与删除

```

142 Node* insert_after(Node* predecessor, int value) {
143     if (predecessor == nullptr) {
144         throw std::invalid_argument("insert_after requires a predecessor");
145     }
146
147     Node* node = new Node{value, predecessor->next};
148     predecessor->next = node;
149     if (tail_ == predecessor) {
150         tail_ = node;
151     }
152     ++length_;
153     return node;
154 }
155
156 bool erase_after(Node* predecessor) {
157     if (predecessor == nullptr || predecessor->next == nullptr) {
158         return false;
159     }
160
161     Node* removed = predecessor->next;
162     predecessor->next = removed->next;
163     if (tail_ == removed) {
164         tail_ = predecessor;
165     }
166     delete removed;
167     --length_;
168     return true;
169 }

```

插入先令新节点继承旧后继，再让前驱指向新节点；若交换这两步，新节点会把自己接成错误后继或丢失旧链。删除先保存目标节点，跨过它，再修复 `tail_`、释放节点并递减长度。这里的  $O(1)$  只从传入合法 `predecessor` 开始，不包含寻找前驱。

### 7.3 按值删除：Locate 与 Update 在代码中也必须分层

Listing 4: 按值删除第一次出现的节点

```

171 bool erase_first(int value) {
172     Node* predecessor = nullptr;
173     Node* current = head_;
174     while (current != nullptr && current->value != value) {
175         predecessor = current;
176         current = current->next;
177     }
178     if (current == nullptr) {
179         return false;
180     }
181
182     if (predecessor == nullptr) {
183         head_ = current->next;
184         if (tail_ == current) {
185             tail_ = nullptr;
186         }
187         delete current;
188         --length_;
189         return true;
190     }
191     return erase_after(predecessor);
192 }

```

循环负责 Locate，最坏访问全部  $n$  个节点；找到后才进入头节点分支或复用 `erase_after`。删除唯一节点时必须同时把 `head_` 和 `tail_` 置空，这正是“边界属于合法状态集合”的代码证据。

### 7.4 不变量检查与测试不是生产操作的成本

伴随实现提供 `invariant()`：检查空表字段一致性、环、可达节点数、尾节点身份及“尾节点的 `next` 为空”。它用于教学验证和测试，不意味着每次生产操作都必须额外遍历全表。

最小断言测试位于 `code/ds01_linear_list_test.cpp`，覆盖顺序表扩容与非法位置、链表空表、已知前驱插入、首尾删除和删除至空表。测试的关键部分直接验证抽象结果和结构不变量：

Listing 5: 链表边界与不变量断言

```

35 void test_singly_linked_list() {
36     SinglyLinkedList list;
37     assert(list.empty());
38     assert(list.invariant());
39
40     bool rejected = false;
41     try {
42         list.insert_after(nullptr, 1);
43     } catch (const std::invalid_argument&) {
44         rejected = true;

```

```
45     }
46     assert(rejected);
47
48     list.push_back(7);
49     list.push_back(2);
50     list.push_back(9);
51     auto* predecessor = list.find_first(2);
52     list.insert_after(predecessor, 5);
53     assert((list.values() == std::vector<int>{7, 2, 5, 9}));
54     assert(list.invariant());
55
56     assert(!list.erase_after(list.tail()));
57     assert(list.invariant());
```

编译验证命令为：

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
    code/ds01_linear_list_test.cpp -o /tmp/ds01_test
/tmp/ds01_test
```

做题时的第一判断

题目给“第  $k$  个位置/下标”时，先判断表示能否直接映射位置；题目给“某个关键字”时，先把查找成本单列；题目给“节点指针/前驱指针”时，才可以把链式局部修改写成  $O(1)$ 。不要让操作名称替代输入契约。

## 8 空、首、尾与重复值不是补丁

| 边界状态 | 顺序表检查                     | 链式表示检查                           |
|------|---------------------------|----------------------------------|
| 空表   | $length = 0$ ，不能访问 $A[0]$ | $head = null$ ， $tail$ 也必须一致     |
| 单元素  | 删除后长度变为 0                 | 删除后 $head = tail = null$         |
| 头部   | 插入/删除会移动或更新首位置            | 更新 $head$ ，不能丢掉旧链                |
| 尾部   | 追加可能触发扩容                  | 更新 $tail$ ，保持 $tail.next = null$ |
| 重复值  | 明确删第一次、全部还是指定位置           | 不能用“找到一个”替代题目契约                  |
| 越界   | 先判 $0 \leq k \leq length$ | 先判指针/位置可达                        |

最常见的错误推理

“链表插入一定比数组快”忽略了定位；“数组访问一定  $O(1)$ ”忽略了按值查找；“长度字段存在就一定正确”忽略了每次分支更新；“删除节点后把指针设空就完成了删除”忽略了前驱、 $head/tail$  和回收责任。

9 概念边界：把相似说法还原成判据

| 概念 A  | ≠ 概念 B  | 真正区别与题面信号              | 混淆后果           |
|-------|---------|------------------------|----------------|
| 逻辑线性表 | ≠ 顺序表   | 前者是关系/操作契约，后者是连续表示     | 把一种实现当成 ADT 定义 |
| 位置已知  | ≠ 关键字已知 | 位置可直接进入更新，关键字仍需定位      | 漏算查找成本         |
| 节点已定位 | ≠ 节点可达  | 有指针只说明能沿链走，不说明已拿到前驱    | 把删除写成错误的 O(1)  |
| 长度    | ≠ 容量    | 有效元素数量 vs 已分配槽位数量      | 扩容/越界条件写反      |
| 访问时间  | ≠ 操作总成本 | 单次寻址/比较 vs 定位、移动、维护的总和 | 复杂度结论失真        |

10 做题控制协议与压缩复原

遇到线性表题，按以下顺序写草稿：

- 1. 写清逻辑操作的输入契约：位置、关键字、节点还是前驱；
- 2. 画表示：连续下标或节点/指针，并标出 length、capacity、head、tail；
- 3. 写操作前不变量；
- 4. 只执行最小状态更新，再补边界分支；
- 5. 用一次遍历或边界地址验证逻辑次序、可达性和长度；
- 6. 最后分别报告定位、局部更新、移动/跳转、额外空间的成本。

一页压缩

忘掉具体代码后，只需复原五个问题：对象是什么？输入契约是什么？数据在哪里？操作后必须保持什么？哪一项成本被表示换走了？连续表示用地址映射换按位访问；链式表示用指针关系换局部插删。所有“快/慢”判断都必须回到操作输入和不变量。

11 全科坐标与跨册交接

Map Coordinate: Representation / Operation / Invariant / Cost

DS01 把 Atlas 的统一语言落到线性关系：输出“逻辑位置—物理地址/指针—可执行操作—合法状态—成本向量”。DS02 复用顺序/链接表示实现端点限制；DS09 复用有序线性存储讨论查找与索引代价；DS-B01 只把链式/数组状态作为 frontier 容器调用。

线性表的本册停止点是“表示如何支撑操作”。一旦题目转而问栈/队列为何产生时间秩序、树/图如何展开关系或索引如何维护有序查询，应交给相应 Owner。

12 来源处理

本册吸收归档《数据结构 MOC》和《算法时间复杂度：统一分析框架》的对象—表示—操作—成本语言，并将算法笔记中的列表/队列/字典 API 仅作为实现例子，不把 Python 容器行为提升为 408 数据

结构定义。扩容摊还、缓存局部性和泛型容器保留为 Extension；栈、队列、串、树、图、Hash 与排序分别由其他 Topic Own。